

level 2 data collections in zarr

CNG Forum 2026 · Thu, Oct 08, 2026



Tom Nicholas · Earthmover



Sean Harkins · Development Seed



OUTLINE

what we'll cover

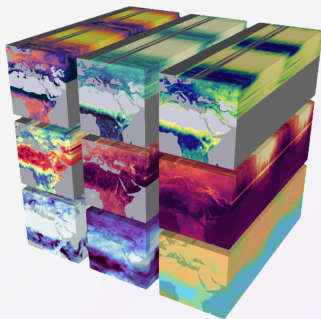
1. Features of an ideal cloud-native data system
2. Level 2 vs Level 3 today
3. Limitations of COG + STAC
4. Idea: Icechunk DataCollections
5. (Sean) Prototypes and future work



CNG FORUM 2025

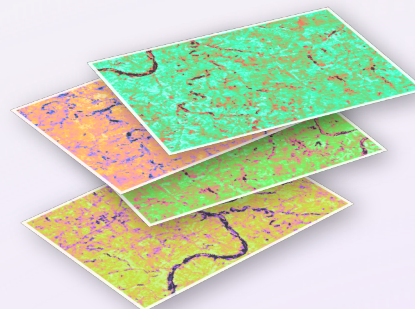
last year: zarr vs cog

Lots of debate last year, and the rough consensus was:



LEVEL 3 · GRIDDED DATACUBES

Zarr wins



LEVEL 2 · INDIVIDUAL SCENES

COG + STAC wins

but similar aims: so why the split?



DESIRED FEATURES

icechunk-zarr works great! (for level 3)

Property	Icechunk-Zarr
Single endpoint  	✓
Query by coordinates  	✓
Scalable  	✓
Serverless  	✓
Uncoordinated reads  	✓
Uncoordinated writes 	✓

 i.e. "cloud-optimized"

 from Zarr

Property	Icechunk-Zarr
Arbitrary N-D schemas 	✓
Domain-agnostic 	✓
ACID transactions and consistency 	✓
Versioning, time travel, branches 	✓
Schema evolution 	✓
Zero-copy ingestion ("virtual chunks") 	✓

 from Xarray

 from Icechunk



LEVEL 2 VS LEVEL 3

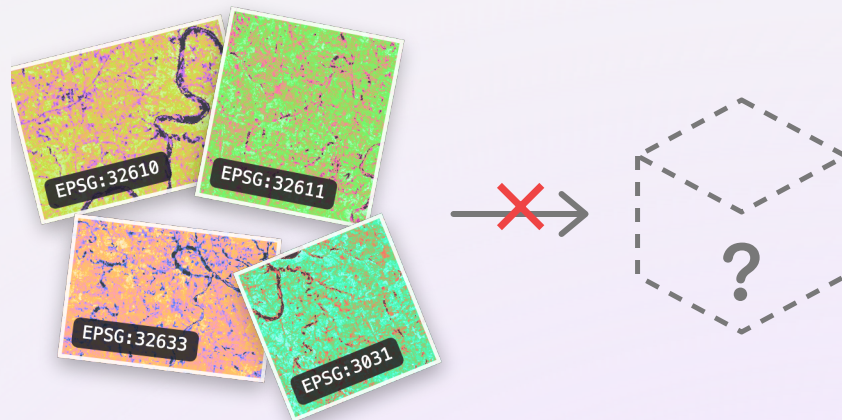
but level 2 in zarr is unsolved

- You **cannot** think of Level 2 data as one datacube
- Problem: **many arrays, no shared coordinate system**
- So geospatial folks use many separate COGs, then index them with STAC
 - We also see anti-patterns: many small Zarrs, or one Zarr with too many groups

Important

This problem is **not** specific to geospatial 🌍 !

- 🔬 Bioimaging: millions of OME-TIFFs + an index.
- 🔭 Astronomy: many images as separate FITS files.
- 🔗 Fusion: many Zarr stores + a Parquet index.



different footprints and CRSs: nothing shared to stack along



FEATURE COMPARISON

stac + cog solves some problems, but not all

Property	Icechunk-Zarr	STAC + COG
Single endpoint 	✓	✓
Query by coordinates 	✓	✓
Scalable 	✓	✓
Serverless 	✓	● ¹
Uncoordinated reads 	✓	✓
Uncoordinated writes	✓	● ²

 i.e. "cloud-optimized"

Property	Icechunk-Zarr	STAC + COG
Arbitrary N-D schemas	✓	✗
Domain-agnostic	✓	✗
ACID transactions and consistency	✓	✗
Versioning, time travel, branches	✓	✗
Schema evolution	✓	✗
Zero-copy ingestion	✓	● ³

- ¹ only with stac-geoparquet (no STAC API server)
- ² only if writing to separate COG files
- ³ only if data already Cloud-Optimized (e.g. COG/Zarr)



IN PRACTICE

cog's limitations cause real problems

Consistency failures

During operational updates, the catalog and the data can disagree

Rigid data layouts

No hyperspectral bands, no ensemble members, no time dimension

Query-optimized chunking

Can't always rechunk for access patterns, e.g. timeseries

Expensive modifications

Renaming one variable in a 1TB TIFF means downloading and re-uploading 1TB

Domain-specific

Microscopists have near-identical requirements, but COG + STAC are geo-only

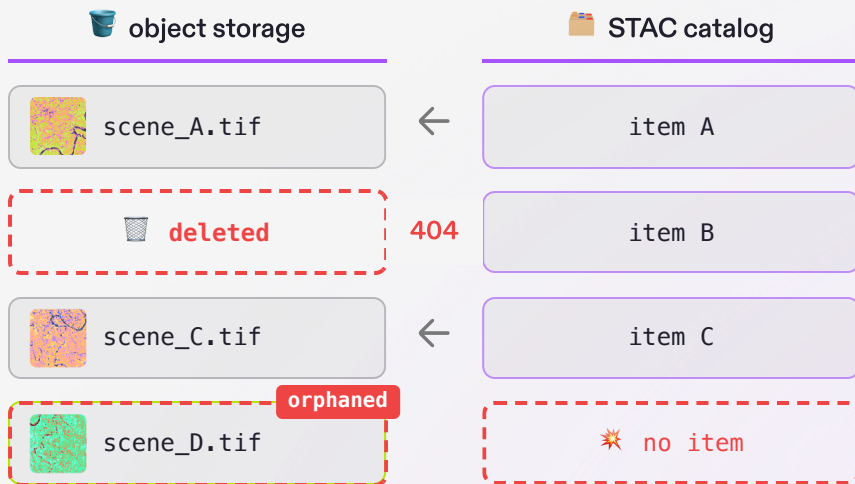
Existing archives

Can't reference non-cloud-optimized data, e.g. non-COG TIFFs



METADATA SEPARATE FROM DATA

disjoint systems: no consistency guarantees!



Data and metadata live in **two systems**, linked only by href s.

① Producer writes a new COG...

💥 ...but the job crashes before updating STAC → **orphaned data**, invisible to search

② Scene B is reprocessed and the old COG deleted → **dangling link**

Nothing ties the two together transactionally → **consistency problems**



SINGLE FILE FORMAT = DEAD END

these limitations are inherent to cog

A single-file container format can never support key features in object storage, since object storage has no edit-in-place.

- Prevents truly uncoordinated writes
- Prevents cheap updates → no schema evolution, no versioning
- **Prevents consistency(!!)** using the serverless mechanism
Icechunk uses (conditional puts)
- Virtual references inherently involve multiple files

TIFF as a container constrains the schema

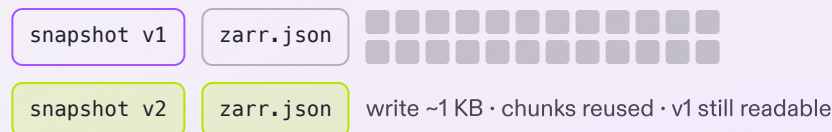
GeoTIFF + STAC are domain-specific

 rename one variable...

COG one 1 TB object



Icechunk-Zarr many small objects

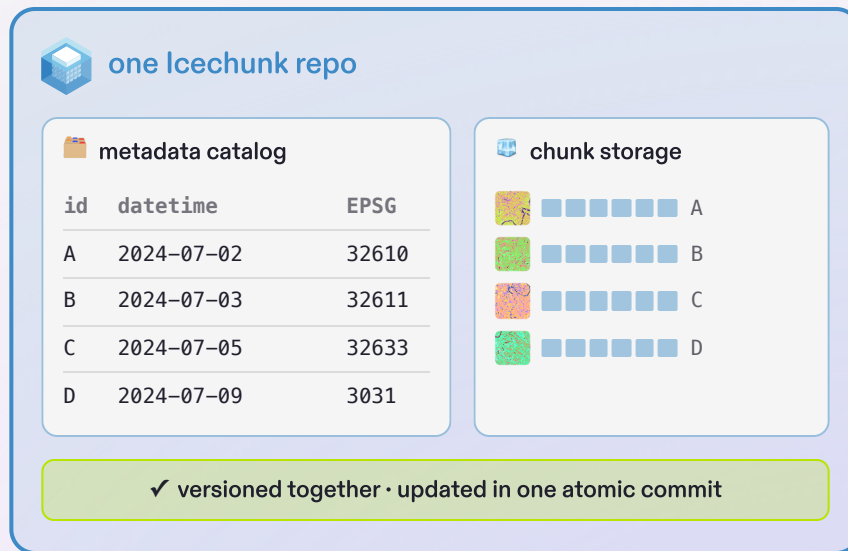




IDEA: ICECHUNK DATACOLLECTIONS

better: many images + catalog, in one repo

- What if we put **many, many images** (any shapes, any CRSs) into one Icechunk repo...
 - ...alongside a 1D, **STAC-like metadata table** to search them?
- Chunks and catalog are versioned together: **ACID, consistency, versioning and virtual chunks, for free**
- And it's domain-agnostic: any set of related arrays





developmentSEED

SEAN HARKINS · DEVELOPMENT SEED

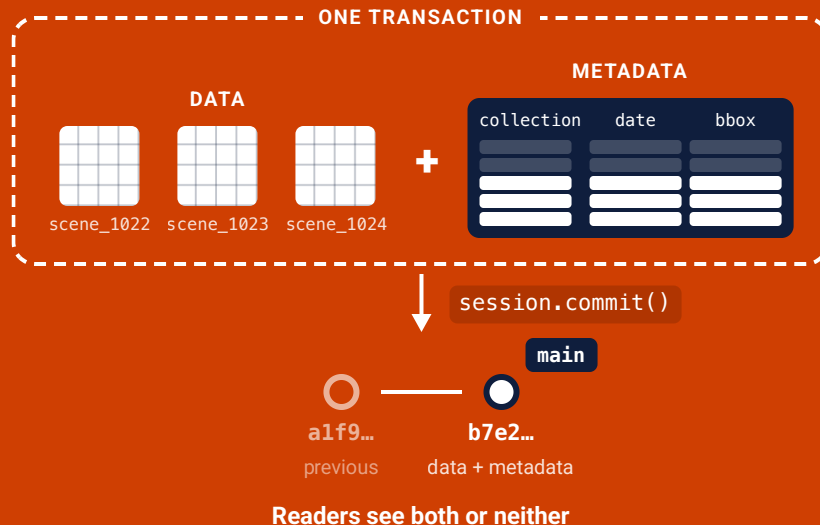
Storing data and STAC-like metadata together

DESIGN

What Do We Want In A ~~Level 2 Zarr System~~ Array Database?

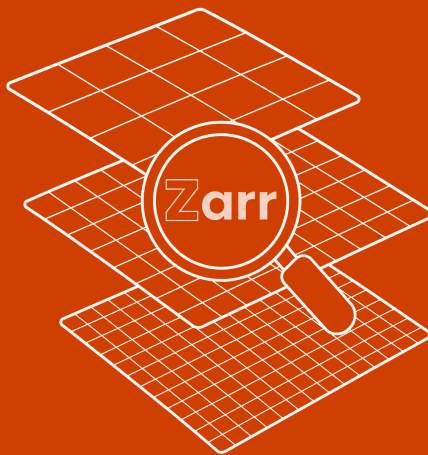
At this point we need to take a step back and realize that we're trying to build database technology that needs to solve these problems.

1. Storing columnar metadata.
2. Querying using industry standard tooling.
3. Supporting transactions for **data and metadata** together.
4. Scaling to **billions?** of arrays.



INITIAL ATTEMPT

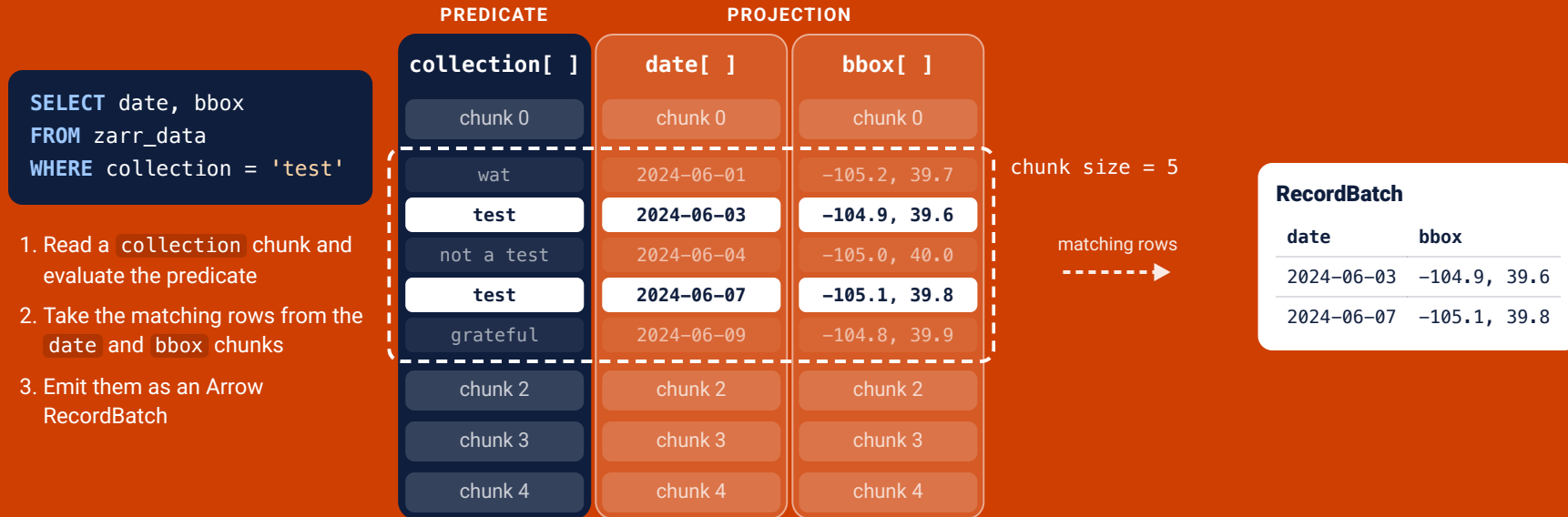
Most of the "metadata" we're discussing can be modeled as an Arrow schema. So as an initial experiment we thought **What if we could store Arrow like data in Icechunk?**



Zarr-Datafusion-Search

ZARR-DATAFUSION-SEARCH

Datafusion let's us build queryable database-like systems from any backend that can emit Arrow RecordBatches. With this we can represent columnar formats like Parquet using a series of 1-D Zarr arrays.



ZARR-DATAFUSION-SEARCH

Pros and cons

Pros

Only requires **Icechunk** and a compliant **DataFusion table provider**.

Materialized **R-tree and B-tree indexes** can be stored as Zarr arrays and used in the query pipeline, which can be much more efficient than common Parquet engine pushdown optimizations.

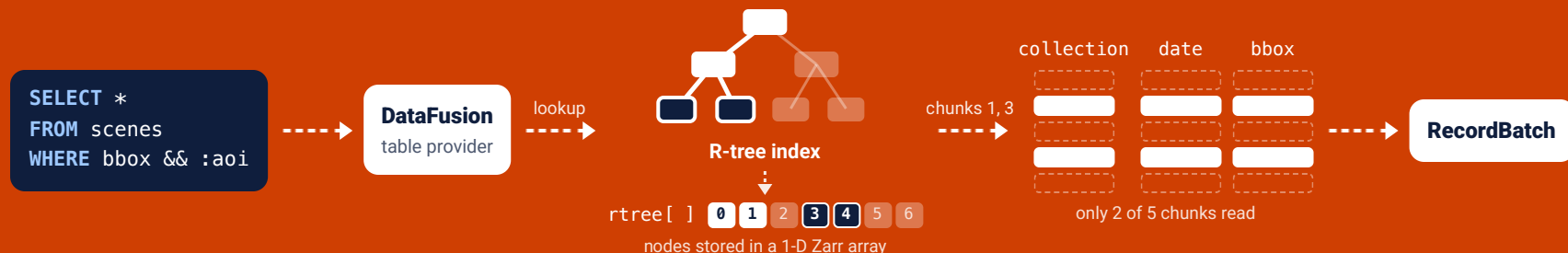
Writing data only requires an **Icechunk-compatible Zarr client**.

Cons

A very **custom solution** that doesn't leverage other great industry tools.

Writers need to coordinate writing their metadata "columns" so that "rows" are **chunk aligned**.

Variable-length dtypes have poor decoding performance in Zarr.



ICECHEST

The best of both worlds?

What if we could take advantage of the best parts of an **array store** like Icechunk and an **open table format** like Iceberg?



Icechunk

Array data

+



Apache Iceberg

Tabular metadata

=



icechest

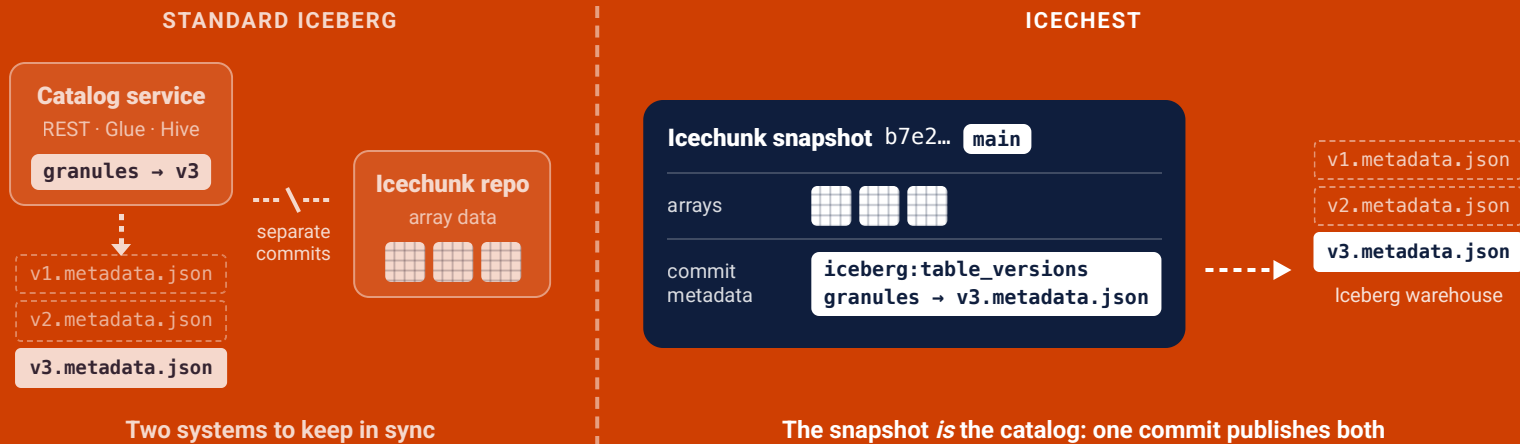
arrays + tables

In one Icechunk commit

ICECHEST

How it works

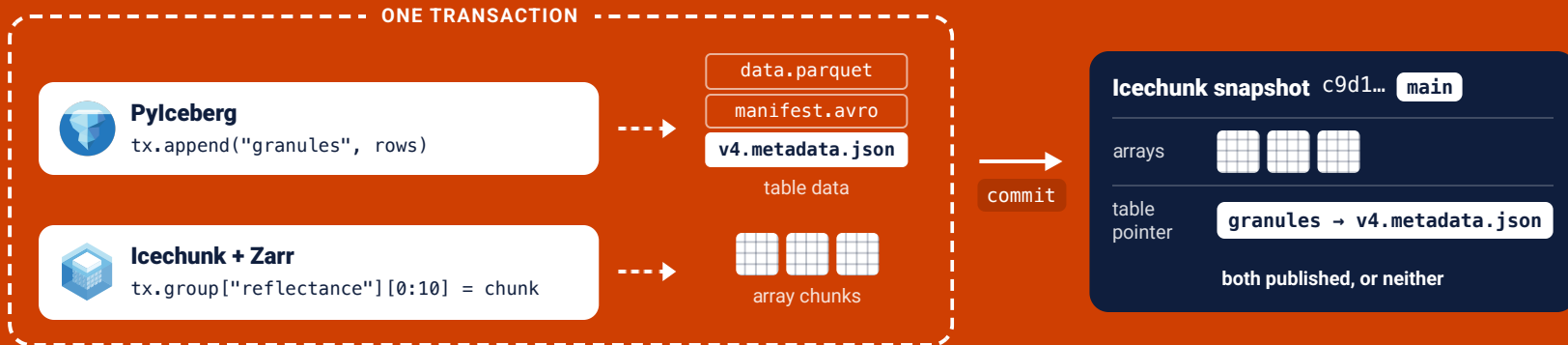
Iceberg normally relies on a **catalog service** to track the current `metadata.json`. Instead of a catalog service, Icechest stores that pointer in **Icechunk commit metadata**.



ICECHEST PROS

Standard tooling, one commit

Rather than reinventing the wheel like we did with Zarr-Datafusion-Search, Icechest lets us use **standard tooling** for writing and reading both **columnar data** and **array data**.



ICECHEST CONS

Table maintenance is on us

A catalog service normally handles table maintenance. In Icechest, Iceberg versions are pinned by **Icechunk snapshots**, which have their **own lifecycle**, so we have to manage maintenance ourselves.

STANDARD ICEBERG

Catalog service

- ✓ Data compaction
- ✓ Metadata compaction
- ✓ Orphan-file removal
- ✓ Snapshot expiry

Judged from the table's own history

ICECHEST

tag v1.0

a1f9...

v2.metadata.json

b7e2...

v3.metadata.json

main

c9d1...

v4.metadata.json



Iceberg's expiry only sees **v4**, so it would delete files that **tag v1.0** still needs

We'll need to run these ourselves, with Icechunk refs as the source of truth

Data compaction

Metadata compaction

Orphan-file removal

Snapshot expiry

ICECHEST CONS

Icechunk node scaling

Level 3 data may have billions of chunks, but in **one array**. Level 2 data means **many discrete groups and arrays**. Virtualizing the full HLS archive into Icechest takes a mind-boggling number of **Zarr nodes**.

LEVEL 3



1 array node
billions of chunks

LEVEL 2 (HLS)

```

/
├── HLS.S30.T10SEG.2024001/
│   ├── B01/ multiscales
│   │   ├── 0
│   │   ├── 1
│   │   └── 2
│   ├── B02/ ...
│   └── Fmask/ ...
├── HLS.L30.T10SEG.2024003/ ...
└── ... 38 M granules
    
```

181–217 nodes per granule
group/ array

collection	granules	nodes/granule	total nodes
HL30	16,102,070	181	2.91 B
HLSS30	22,051,832	217	4.79 B
combined	38,153,902		7.70 B

Commit memory and time grow with the **total nodes in the store** ([icechunk#2449](#)): one year of HLS (~318 M nodes) extrapolates to **~499 GiB** and **~5.7 min** per commit.

level 2 Icechunk data collections

what do you think?



Tom Nicholas · Earthmover



Sean Harkins · Development Seed



BONUS

common objections

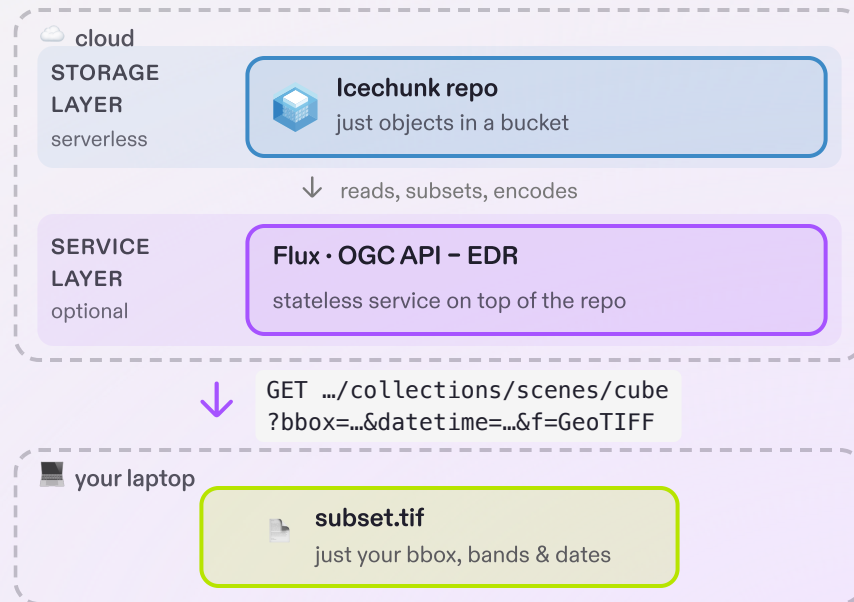


SINGLE-FILE DOWNLOAD

"but i like downloading a single file"

sure — but that needn't dictate the storage layout

- It's reasonable to want a file on your local filesystem
- But we're free to transform the data **any way we like** on its journey from the datacentre to your laptop
- So use a service (e.g. Flux, via OGC API – EDR) to subset the store and assemble a GeoTIFF on request
- A file that never existed on disk — **the perfect COG for your immediate need**





COMPATIBILITY

"but many applications understand tiff!"

yes — *filesystem* applications

- We can **reconstruct GeoTIFFs on demand** for any local program (e.g. ArcGIS)
- It's genuinely impressive that macOS Preview can open a COG — a feat of format stability
 - But no one points Preview at object storage
- Any application that reads TIFFs **from object storage** can be taught to read Zarr (it's all just range requests)
- Compatibility via adapter layer is a **much easier problem** than consistency - vibe code it!

